Documentation complète — installeur OL9.6 custom

Référence exhaustive de chaque fonction, chaque commande système appelée, et chaque constante du projet. Organisé module par module dans l'ordre d'exécution (`main.py`).

Aucune classe dans ce projet : tout est écrit sous forme de fonctions simples. C'est volontaire — c'est un outil de provisioning "one-shot" (on l'exécute une fois par poste, pas d'état à faire vivre entre plusieurs appels), donc pas besoin d'objets avec un cycle de vie. Une fonction qui prend des paramètres et renvoie un résultat est plus simple à lire, à tester isolément, et à documenter qu'une hiérarchie de classes.

Sommaire

1. config.py
2. rpm_installer.py
3. libreoffice_setup.py
4. accounts.py
5. shortcuts.py
6. gnome_extensions.py
7. opt_permissions.py
8. java_setup.py
9. password_policy.py
10. deploy_bundle.py
11. install.py
12. main.py

---

config.py

`load_config(path="config.json")`

Syntaxe

```
load_config(path="config.json") -> dict
```

Utilité Lit un fichier JSON et le transforme en `dict` Python. C'est le seul point d'entrée vers `config.json` dans tout le projet — aucun autre module ne lit directement ce fichier.

Exemple

```
from config import load_config cfg = load_config() # lit "config.json" dans le dossier courant cfg = load_config("/etc/mon.json") # ou un chemin explicite
```

```
print(cfg["rpm_dir"]) # "/opt/install/rpms"
```

Erreurs à ne pas faire - Appeler `load_config()` depuis un dossier qui n'est pas celui du projet : le chemin par défaut `"config.json"` est **relatif au dossier de travail courant** (`cwd`), pas au dossier du script. Si tu lances `sudo python3 /tmp/scriptInstallApps/main.py` depuis `/home/naval/`, il cherchera `/home/naval/config.json`, pas celui du projet. Toujours lancer les scripts depuis le dossier qui contient `config.json`. - Oublier qu'un JSON malformé (virgule en trop, guillemets simples au lieu de doubles) fait planter `json.load` avec une `json.decoder.JSONDecodeError` peu lisible. Toujours valider le JSON après édition manuelle (ex. `python3 -m json.tool config.json`).

Pourquoi ainsi `json` est dans la bibliothèque standard (aucune dépendance externe à installer sur une machine hors ligne), le format est facile à éditer à la main et à versionner dans git. On a choisi JSON plutôt que YAML précisément pour éviter d'installer PyYAML sur une cible qui n'a pas accès à internet.

---

rpm_installer.py

`find_rpms(rpm_dir)`

Syntaxe

```
find_rpms(rpm_dir: str) -> list[str]
```

Utilité Liste tous les fichiers `*.rpm` présents dans `rpm_dir`. Lève une erreur si le dossier n'existe pas ou s'il est vide, plutôt que de renvoyer silencieusement une liste vide.

Exemple

```
fichiers = find_rpms("/opt/install/rpms") # ->
["/opt/install/rpms/jq-1.6-1.el9.x86_64.rpm", "/opt/install/rpms/xterm-...", ...]
```

Erreurs à ne pas faire - Passer un chemin qui existe mais qui pointe vers un fichier (pas un dossier) : `os.path.isdir` renverra `False` et tu auras un `FileNotFoundError` avec un message "le dossier n'existe pas" un peu trompeur (le fichier existe, ce n'est juste pas un dossier). - Croire que `glob.glob("*.rpm")` est récursif : il ne descend pas dans les sous-dossiers. Si les RPM sont rangés dans des sous-dossiers par catégorie, `find_rpms` ne les verra pas.

Pourquoi ainsi `glob` fait exactement ce qu'on veut ici (motif simple à plat, pas de recherche récursive) sans dépendance externe. Le garde-fou "dossier vide" existe parce qu'un dossier `/opt/install/rpms` vide n'est presque toujours pas une intention volontaire mais un oubli de déploiement (bundle pas encore copié) — mieux vaut planter tout de suite avec un message clair que de laisser `install_rpms([])` s'exécuter silencieusement sur une liste vide.

`calculate_sha256(file_path)`

Syntaxe

```
calculate_sha256(file_path: str) -> str
```

Utilité Calcule l'empreinte SHA256 d'un fichier et la renvoie en hexadécimal (chaîne de 64 caractères), pour vérifier son intégrité.

Exemple

```
empreinte = calculate_sha256("/opt/install/rpms/jq-1.6-1.el9.x86_64.rpm") # ->
"3f786850e387550fdab836ed7e6dc881de23001b3d5b1a5..."
```

Erreurs à ne pas faire - L'ouvrir en mode texte (`open(file_path)` sans `"rb"`) : un RPM est un fichier binaire, l'ouvrir en mode texte casserait l'encodage sur certains octets et donnerait un hash complètement faux (ou une exception `UnicodeDecodeError` selon la plateforme). - Appeler cette fonction sur un très gros fichier en boucle serrée sans réfléchir à la mémoire : `f.read()` charge **tout le fichier en RAM d'un coup**. Pour des RPM (quelques Mo à quelques centaines de Mo), c'est sans risque ; pour des fichiers de plusieurs Go, il faudrait lire par blocs (`f.read(65536)` en boucle avec `hash.update()`).

Pourquoi ainsi `hashlib` est stdlib, SHA256 est le standard actuel pour ce genre de vérification (`SHA256SUMS` est le nom de fichier conventionnel généré par `sha256sum` sous Linux). Lire le fichier entier d'un coup plutôt que par blocs simplifie le code et reste largement acceptable vu la taille des fichiers concernés ici.

`parse_sha256sums(sums_file)`

Syntaxe

```
parse_sha256sums(sums_file: str) -> dict[str, str]
```

Utilité Lit un fichier au format `sha256sum` (une ligne par fichier : `empreinte nom_fichier`) et le transforme en dict `{nom_fichier: empreinte}` pour un lookup facile.

Exemple

```
# Contenu de SHA256SUMS : # 3f786850e387550fdab836ed7e6dc881de23001b
jq-1.6-1.el9.x86_64.rpm # a94a8fe5ccb19ba61c4c0873d391e987982fbbd3 xterm-...
empreintes = parse_sha256sums("/opt/install/rpms/SHA256SUMS") # ->
{"jq-1.6-1.el9.x86_64.rpm": "3f786850...", "xterm-...": "a94a8fe5..."}
```

Erreurs à ne pas faire - Utiliser `ligne.split(" ", 1)` au lieu de `ligne.split(None, 1)` : `sha256sum` sépare l'empreinte et le nom de fichier par **deux espaces** (ou un espace + un indicateur de mode), pas un seul. `split(None, 1)` découpe sur n'importe quelle suite d'espaces/tabulations, ce qui gère les deux cas (`"empreinte fichier"` et `"empreinte *fichier"`) sans se soucier du nombre exact d'espaces. - Oublier le `.lstrip("*")` : le `*` avant le nom de fichier signifie "mode binaire" chez `sha256sum` — si on ne le retire pas, la clé du dict sera `"*jq-1.6-1.el9.x86_64.rpm"` au lieu de `"jq-1.6-1.el9.x86_64.rpm"`, et `verify_checksums` ne retrouvera jamais la correspondance (tous les fichiers ressortiront comme "absents de SHA256SUMS").

Pourquoi ainsi Un dict donne un lookup en O(1) par nom de fichier dans `verify_checksums`, plutôt que de reparcourir une liste à chaque fichier RPM (O(n) par recherche, O(n²) au total). Vu que le nombre de RPM reste modeste ici, la différence de performance est négligeable, mais c'est la structure de données naturelle pour "je veux retrouver une valeur à partir d'une clé".

`verify_checksums(rpm_dir, sums_file=None)`

Syntaxe

```
verify_checksums(rpm_dir: str, sums_file: str | None = None) -> list[str]
```

Utilité Combine les trois fonctions précédentes : liste les RPM, lit `SHA256SUMS`, compare chaque empreinte calculée à l'empreinte attendue. Renvoie la liste des chemins RPM si tout est bon, lève une `ValueError` sinon.

Exemple

```
rpm_files = verify_checksums("/opt/install/rpms") # -> ["/opt/install/rpms/jq-...",
"/opt/install/rpms/xterm-..."] si tout correspond
verify_checksums("/opt/install/rpms") # ValueError: Vérification SHA256 échouée: # -
jq-1.6-1.el9.x86_64.rpm: empreinte différente (attendu ..., obtenu ...) # - xterm-...:
absent de SHA256SUMS
```

Erreurs à ne pas faire - Passer un `sums_file` qui ne correspond pas réellement au contenu actuel de `rpm_dir` (ex. un `SHA256SUMS` généré avant d'avoir ajouté de nouveaux RPM dans le dossier) : la fonction lèvera une erreur "absent de SHA256SUMS" pour chaque nouveau fichier, ce qui est le comportement voulu mais peut surprendre si on a oublié de régénérer le fichier après un ajout. - S'arrêter au premier `raise` en pensant corriger un fichier à la fois : la fonction est justement conçue pour accumuler **tous** les problèmes dans `invalides` avant de lever une seule erreur groupée, pour éviter les allers-retours "je corrige un fichier, je relance, j'en découvre un autre".

Pourquoi ainsi Grouper toutes les erreurs en une seule exception (plutôt que `raise` dès le premier fichier suspect) donne une vue complète du problème en un seul run — précieux sur une machine hors ligne où chaque essai/erreur prend du temps (il faut retransférer des fichiers, relancer le script).

`install_rpms(rpm_files)`

Syntaxe

```
install_rpms(rpm_files: list[str]) -> str
```

Utilité Installe la liste de RPM donnée via `dnf`. Lève `PermissionError` si pas root, `RuntimeError` si `dnf` échoue.

Commande système utilisée

```
dnf install -y --disablerepo=* <fichier1.rpm> <fichier2.rpm> ...
```

- `-y` : répond "oui" automatiquement à toute confirmation (le script tourne sans interaction).
- `--disablerepo=*` : désactive **tous** les dépôts configurés (distants). Sans ce flag, une machine avec un accès réseau, même partiel ou lent, ferait attendre `dnf` indéfiniment en essayant de contacter des dépôts injoignables. Avec ce flag, `dnf` ne peut résoudre les dépendances qu'entre les fichiers explicitement passés en argument.

Exemple

```
rpm_files = verify_checksums("/opt/install/rpms") sortie = install_rpms(rpm_files)
print(sortie) # stdout de dnf : liste des paquets installés, tailles, etc.
```

Erreurs à ne pas faire - Ajouter `--allowerasing` sans comprendre pourquoi `dnf` en a besoin : ce flag autorise `dnf` à **désinstaller** des paquets déjà présents pour résoudre la transaction. Si le dossier `rpm_dir` contient des doublons de composants système (mauvaise version d'une lib déjà installée), `dnf` peut proposer de désinstaller des paquets protégés (`dnf`, `yum` eux-mêmes) — signe qu'il faut nettoyer le lot de RPM, pas forcer le flag. Voir l'incident documenté dans `INSTALL.md`. - Ajouter `--alldeps` au moment du **téléchargement** (`dnf download`) depuis un dépôt **en ligne** : ce flag force le téléchargement de **toutes** les dépendances, y compris celles déjà satisfaites localement, et va chercher la dernière version disponible sur le dépôt distant — ce qui peut ramener des centaines de paquets système non désirés (`oraclelinux-release` compris). `install_rpms` elle-même n'a pas ce problème (elle ne télécharge rien), mais le lot qu'on lui donne doit avoir été construit sans `--alldeps` depuis un dépôt en ligne. Exception : le lot LibreOffice est téléchargé avec `--alldeps` depuis l'**ISO 9.6** montée en dépôt local, dont les versions sont exactement celles de l'OS vierge.

Usage en ligne de commande `rpm_installer.py` peut aussi se lancer seul sur n'importe quel dossier de RPM (avec son `SHA256SUMS`), sans passer par `main.py` :

```
sudo python3 rpm_installer.py /opt/install/libreoffice
```

Le bloc `if __name__ == "__main__":` lit le chemin dans `sys.argv[1]` (le premier mot tapé après le nom du script), appelle `verify_checksums` puis `install_rpms`. C'est ce qui permet de déployer LibreOffice hors bundle sur un poste déjà en production.

Pourquoi ainsi `subprocess.run(..., capture_output=True, text=True)` plutôt que `os.system()` : on récupère `stdout`/`stderr` en tant que texte pour construire un message d'erreur exploitable, et on évite les problèmes d'injection shell qu'`os.system()` peut introduire si un chemin de fichier contient des caractères spéciaux (`os.system` passe par un shell, `subprocess.run` avec une liste d'arguments non).

---

libreoffice_setup.py

Ce module installe la suite LibreOffice depuis un dossier local de RPM. Il **réutilise** `rpm_installer.py` (même vérification SHA256, même `dnf install`) et y ajoute un contrôle final. Il peut tourner seul, hors `install.py` et `main.py`, comme `accounts.py`.

Constantes `DEFAULT_RPM_DIR` et `REQUIRED_PACKAGES`

Syntaxe

```
DEFAULT_RPM_DIR: str = "/opt/install/libreoffice" REQUIRED_PACKAGES: list[str] = [
"libreoffice-core", "libreoffice-writer", "libreoffice-calc", "libreoffice-impress", ]
```

Utilité `DEFAULT_RPM_DIR` est le dossier où le bundle dépose les RPM de LibreOffice (utilisé quand on ne précise aucun dossier). `REQUIRED_PACKAGES` est la liste des paquets qui doivent être présents une fois l'installation terminée : c'est ce que le contrôle final vérifie.

Erreurs à ne pas faire - Ajouter un paquet à `REQUIRED_PACKAGES` sans l'avoir mis dans le lot téléchargé : le contrôle final échouera à chaque installation, alors que `dnf` aura réussi. - Oublier de mettre à jour cette liste si tu changes la composition du lot LibreOffice (ex. retirer Impress) : le contrôle réclamera un paquet qui n'existe plus.

Pourquoi ainsi On ne contrôle que le cœur de la suite (`core`, `writer`, `calc`, `impress`) plutôt que les 20 paquets du lot : si ces quatre-là sont présents, la suite est utilisable, et la liste reste courte à maintenir.

`verify_libreoffice()`

Syntaxe

```
verify_libreoffice() -> None
```

Utilité Vérifie que chaque paquet de `REQUIRED_PACKAGES` est bien installé. Lève une `RuntimeError` qui affiche la sortie de `rpm` si l'un d'eux manque.

Commande système utilisée

```
rpm -q libreoffice-core libreoffice-writer libreoffice-calc libreoffice-impress
```

`rpm -q` (q pour *query*) interroge la base RPM locale : il affiche la version de chaque paquet installé, et renvoie un code de sortie non nul dès qu'**un seul** est absent. C'est ce code que Python lit via `resultat.returncode`.

Exemple

```
verify_libreoffice() # Silencieux si tout est installé. # RuntimeError: LibreOffice
incomplet après installation: # libreoffice-impress n'est pas installé <- si un paquet
manque
```

Erreurs à ne pas faire - Croire qu'une installation `dnf` réussie suffit à prouver que le lot est complet : un RPM oublié dans le bundle ne fait pas échouer `dnf`, il n'est simplement jamais installé. C'est exactement ce qui est arrivé sur le terrain, et c'est pour ça que ce contrôle existe.

Pourquoi ainsi `rpm -q` plutôt que chercher l'exécutable `soffice` sur le disque : on interroge la source de vérité (la base RPM), sans dépendre du chemin où l'exécutable est installé.

`install_libreoffice(rpm_dir=DEFAULT_RPM_DIR)`

Syntaxe

```
install_libreoffice(rpm_dir: str = "/opt/install/libreoffice") -> None
```

Utilité Enchaîne trois étapes : `rpm_installer.verify_checksums(rpm_dir)` (les empreintes SHA256 correspondent), `rpm_installer.install_rpms(...)` (installation `dnf`), puis `verify_libreoffice()` (contrôle final).

Exemple

```
install_libreoffice() # dossier par défaut
install_libreoffice("/media/usb/libreoffice") # autre dossier
```

En ligne de commande, depuis le dossier du projet :

```
sudo python3 libreoffice_setup.py [/chemin/vers/dossier_rpm]
```

Sans argument, le dossier par défaut est utilisé. Plus d'un argument affiche l'usage et s'arrête.

Erreurs à ne pas faire - Lancer ce script sur un poste déjà déployé sans essai à blanc préalable (`dnf install --assumeno ...`, voir `INSTALL.md`) : le lot vient de l'ISO 9.6, alors que ces postes ont reçu quelques paquets plus récents. Il faut s'assurer que `dnf` ne propose aucun `Removing` ni `Downgrading`. - Le copier seul sur un poste : il importe `rpm_installer.py`, donc les deux fichiers doivent être dans le même dossier.

Pourquoi ainsi La vérification SHA256 et l'installation sont déléguées à `rpm_installer.py` plutôt que recopiées : un seul endroit à corriger si ce comportement change. Le contrôle `rpm -q` final est la seule logique propre à LibreOffice.

---

accounts.py

`prompt_user_info()`

Syntaxe

```
prompt_user_info() -> dict # -> {"username": str, "password": str, "force_change": bool, "add_to_wheel": bool}
```

Utilité Pose quatre questions à l'opérateur dans le terminal (nom d'utilisateur, mot de passe temporaire, forcer le changement au prochain login, ajout au groupe `wheel`) et renvoie les réponses sous forme de dict, prêt à être passé à `create_user(**infos)`.

Exemple

```
infos = prompt_user_info() # Nom d'utilisateur : jdupont # Mot de passe temporaire : (saisie invisible) # Forcer le changement de mot de passe à la prochaine connexion ? (1 = oui, Entrée = non) : 1 # Ajouter cet utilisateur au groupe wheel (sudo) ? (1 = oui, Entrée = non) : 1 print(infos) # {"username": "jdupont", "password": "...", "force_change": True, "add_to_wheel": True}
```

Erreurs à ne pas faire - Afficher `infos["password"]` dans un `print()` de debug : le mot de passe est en clair dans le dict Python en mémoire (c'est nécessaire pour le passer à `chpasswd`), mais il ne doit **jamais** être loggé, imprimé, ou écrit dans un fichier. Le `__main__` de ce module fait exprès de n'afficher que `username` et `force_change`. - Croire que `getpass.getpass()` fonctionne dans un terminal qui n'est pas un vrai TTY (ex. certains IDE, ou un script lancé via un pipe) : dans ce cas, `getpass` bascule sur un `input()` classique **avec** écho visible, avec un avertissement — pas une erreur silencieuse, mais un piège si on ne le sait pas.

Pourquoi ainsi `getpass` plutôt que `input()` pour le mot de passe : `input()` afficherait la saisie en clair à l'écran (visible si quelqu'un regarde par-dessus l'épaule, ou si le terminal est partagé/enregistré). La boucle `while not username` plutôt qu'une validation après coup : on ne peut pas continuer sans nom d'utilisateur, autant bloquer immédiatement plutôt que de laisser l'opérateur remplir tout le formulaire pour échouer à la fin.

`create_user(username, password, force_change=False, add_to_wheel=False)`

Syntaxe

```
create_user(username: str, password: str, force_change: bool = False, add_to_wheel: bool = False) -> None
```

Utilité Crée un compte local Linux : `useradd` (avec tous les défauts système), `chpasswd` (mot de passe initial), `chage -d 0` si `force_change=True` (le compte devra changer son mot de passe à la prochaine connexion), et `usermod -aG wheel` si `add_to_wheel=True` (accès sudo).

Commandes système utilisées

```
useradd <username> chpasswd # avec "username:password\n" sur stdin chage -d 0 <username> # seulement si force_change=True usermod -aG wheel <username> # seulement si add_to_wheel=True
```

Exemple

```
create_user("jdupont", "TempPass123", force_change=True, add_to_wheel=True) # Crée /home/jdupont, pose le mot de passe, force le renouvellement au prochain login, # et ajoute jdupont au groupe wheel (accès sudo).
```

Erreurs à ne pas faire - Passer le mot de passe en argument de `chpasswd` (ex. `["chpasswd", f"{username}:{password}"]`) au lieu de le passer via `input=` (stdin) : un argument de commande est visible par n'importe quel utilisateur du système via `ps aux` ou `/proc/<pid>/cmdline` pendant toute la durée d'exécution du process, même bref. Passer par stdin évite complètement cette fuite. - Rappeler `create_user` deux fois avec le même `username` : `useradd` échouera avec "l'utilisateur existe déjà" et lèvera une `RuntimeError` — c'est voulu (pas de silent no-op qui masquerait une erreur d'opérateur), mais ça veut dire que ce script n'est pas idempotent sur la création de compte, contrairement à `java_setup.copy_java` par exemple. - Utiliser `usermod -G wheel <username>` (sans le `-a`) en pensant que ça fait la même chose : sans `-a` (append), `-G` **remplace entièrement** la liste des groupes secondaires de l'utilisateur. Sur un compte tout juste créé ça ne change rien puisqu'il n'a pas encore d'autre groupe secondaire, mais c'est une commande dangereuse à retenir telle quelle : sur un compte existant avec d'autres groupes, `-G` sans `-a` les supprimerait tous sauf `wheel`.

Pourquoi ainsi Aucune option `useradd` personnalisée (pas de `-m`, `-s`, `-G`...) : on a délibérément gardé les valeurs par défaut du système (décision prise en cours de route avec l'utilisateur) plutôt que d'ajouter de la configurabilité non demandée. `chage -d 0` plutôt que `passwd -e` : les deux forcent un changement de mot de passe au prochain login, mais `chage -d 0` est la commande "propre" pour manipuler uniquement la date du dernier changement sans toucher à d'autres attributs du compte. `usermod -aG wheel` en étape séparée après `useradd` plutôt que `useradd -G wheel` dès la création : ça garde `useradd` générique (toujours le même appel, sans branchement conditionnel dedans) et isole la logique "wheel ou pas" dans son propre bloc, à l'image de `force_change`.

---

shortcuts.py

`find_desktop_files(src_dir)`

Syntaxe

```
find_desktop_files(src_dir: str) -> list[str]
```

Utilité Identique dans l'esprit à `find_rpms`, mais pour les fichiers `*.desktop`. Lève une erreur si le dossier n'existe pas ou est vide.

Exemple

```
fichiers = find_desktop_files("/opt/install/desktop") # ->
["/opt/install/desktop/cslab.desktop"]
```

Erreurs à ne pas faire - Mettre des fichiers `.desktop` mal formés (sans extension exacte `.desktop`, ex. `monapp.Desktop` avec une majuscule) : `glob.glob("*.desktop")` est sensible à la casse sous Linux, le fichier serait silencieusement ignoré.

Pourquoi ainsi Duplication volontaire avec `find_rpms` plutôt que factorisation en une fonction générique `find_files(dir, pattern)` : les deux fonctions ont des messages d'erreur différents et des usages assez distincts pour que la petite duplication reste plus lisible qu'une abstraction prématurée à deux appelants.

`copy_desktop_files(src_dir, dest_dir="/usr/share/applications/")`

Syntaxe

```
copy_desktop_files(src_dir: str, dest_dir: str = "/usr/share/applications/") ->
list[str]
```

Utilité Copie chaque `.desktop` trouvé vers `dest_dir`, puis pose `chmod 644` dessus (lecture pour tout le monde, écriture pour le propriétaire seulement).

Exemple

```
copies = copy_desktop_files("/opt/install/desktop") # ->
["/usr/share/applications/cslab.desktop"]
```

Erreurs à ne pas faire - Utiliser `shutil.copy2` au lieu de `shutil.copy` en pensant "plus complet donc mieux" : `copy2` copie aussi les métadonnées (dates, permissions) du fichier source. Ici on **veut** poser nous-mêmes les permissions (`0o644`) après coup, donc hériter des permissions de la source (potentiellement 600, ou 777 selon comment le fichier a été préparé) serait contre-productif. - Oublier que `dest_dir` doit déjà exister : `shutil.copy` ne crée pas les dossiars intermédiaires (contrairement à `os.makedirs(..., exist_ok=True)`). Sur une cible standard, `/usr/share/applications/` existe toujours (fourni par le paquet `filesystem`), donc ce n'est pas un problème en pratique, mais ça le serait avec un `dest_dir` personnalisé qui n'existe pas encore.

Pourquoi ainsi `chmod 644` explicite après la copie plutôt que de compter sur un `umask` bien configuré : le `umask` de l'utilisateur qui lance le script (root, via `sudo`) peut varier d'une machine à l'autre ou d'une session à l'autre — poser la permission explicitement rend le résultat prévisible quel que soit le contexte d'exécution.

---

gnome_extensions.py

`ensure_dconf_profile(profile_path="/etc/dconf/profile/user")`

Syntaxe

```
ensure_dconf_profile(profile_path: str = "/etc/dconf/profile/user") -> None
```

Utilité Garantit que le profil dconf "user" contient bien la ligne `system-db:local`, sans laquelle les réglages système qu'on écrit ensuite (via `write_extensions_profile`) seraient tout simplement ignorés par dconf. Ne réécrit rien si c'est déjà en place.

Exemple

```
ensure_dconf_profile() # Crée /etc/dconf/profile/user avec : # user-db:user #
system-db:local # (ou ne fait rien si déjà présent)
```

Erreurs à ne pas faire - Écraser ce fichier sans vérifier son contenu existant s'il a déjà été personnalisé (ex. un admin a ajouté un `system-db:` supplémentaire pour une autre politique) : la fonction vérifie juste la présence de la sous-chaîne `"system-db:local"` avant de réécrire — si le fichier existe déjà avec ce contenu, il n'est pas touché ; s'il existe avec un contenu **différent** qui ne contient pas cette ligne, il sera **totalement écrasé** (pas fusionné). À garder en tête si ce fichier est aussi géré par un autre outil.

Pourquoi ainsi Vérifier avant d'écrire (plutôt que d'écraser systématiquement) rend la fonction idempotente et sûre à rappeler à chaque run de `main.py` sans effet de bord si rien n'a changé.

`write_extensions_profile(extension_uuids, db_dir="/etc/dconf/db/local.d")`

Syntaxe

```
write_extensions_profile(extension_uuids: list[str], db_dir: str =
"/etc/dconf/db/local.d") -> str
```

Utilité Écrit le fichier `00-extensions` dans `db_dir` avec la clé `enabled-extensions` au format GVariant attendu par dconf. Renvoie le chemin du fichier écrit.

Exemple

```
chemin = write_extensions_profile(["dash-to-dock@micxgx.gmail.com",
"appindicatorsupport@rgcjonas.gmail.com"]) # Écrit /etc/dconf/db/local.d/00-extensions
: # [org/gnome/shell] # enabled-extensions=['dash-to-dock@micxgx.gmail.com',
'appindicatorsupport@rgcjonas.gmail.com']
```

Erreurs à ne pas faire - Donner un UUID d'extension incorrect (le vrai UUID GNOME, pas le nom affiché dans l'interface) : dconf acceptera n'importe quelle chaîne sans erreur, mais l'extension ne s'activera jamais puisque le nom ne correspondra à rien d'installé. Toujours vérifier l'UUID exact via `gnome-extensions list` sur une machine où l'extension est déjà installée. - Appeler cette fonction seule sans avoir appelé `ensure_dconf_profile` avant (ou sans avoir fait `apply_dconf` après) : le fichier serait écrit mais totalement sans effet.

Pourquoi ainsi Le nom de fichier `00-extensions` (avec le préfixe numérique `00-`) suit la convention dconf : les fichiers dans `db/local.d/` sont fusionnés dans l'ordre alphabétique, le préfixe numérique sert à contrôler cet ordre si jamais d'autres fichiers de config sont ajoutés plus tard dans ce dossier.

`apply_dconf()`

Syntaxe

```
apply_dconf() -> None
```

Utilité Compile les fichiers texte de `/etc/dconf/db/local.d/` en base binaire dconf. Sans cet appel, les fichiers écrits par `write_extensions_profile` restent de simples fichiers texte inertes.

Commande système utilisée

```
dconf update
```

Exemple

```
apply_dconf() # Après ça, les nouvelles valeurs par défaut sont actives pour toute nouvelle session.
```

Erreurs à ne pas faire - Oublier cet appel après avoir modifié un fichier dans `db/local.d/` à la main (en dehors du script) : c'est l'erreur classique avec dconf, "j'ai édité le fichier mais rien ne change" — dconf ne relit jamais les fichiers texte directement, seulement sa base binaire compilée.

Pourquoi ainsi Fonction séparée plutôt que fusionnée dans `enable_extensions` : ça permet de rappeler `dconf update` isolément si on modifie `db/local.d/` par un autre moyen (à la main, ou un autre script) sans avoir à repasser par toute la logique d'`enable_extensions`.

`enable_extensions(extension_uuids)`

Syntaxe

```
enable_extensions(extension_uuids: list[str]) -> None
```

Utilité Fonction "chef d'orchestre" du module : enchaîne `ensure_dconf_profile` → `write_extensions_profile` → `apply_dconf`. C'est la seule fonction de ce module appelée depuis `main.py`.

Exemple

```
enable_extensions(["dash-to-dock@micxgx.gmail.com"])
```

Erreurs à ne pas faire - Appeler les fonctions internes (`ensure_dconf_profile`, `write_extensions_profile`, `apply_dconf`) directement depuis `main.py` au lieu de passer par `enable_extensions` : ça marche techniquement, mais ça duplique l'ordre d'appel dans deux fichiers différents — si l'ordre doit changer un jour, il faudrait penser à le corriger aux deux endroits.

Pourquoi ainsi Une seule vérification `os.geteuid() != 0` ici, pas dans chacune des trois sous-fonctions : elles écrivent toutes dans des chemins système protégés, donc la vérification root au niveau de la fonction publique suffit ; la dupliquer trois fois n'ajouterait rien.

---

opt_permissions.py

`set_opt_permissions(app_dir)`

Syntaxe

```
set_opt_permissions(app_dir: str) -> None
```

Utilité Pose `chmod 755` sur `app_dir` lui-même, puis récursivement sur tous les sous-dossiers **et** fichiers en dessous.

Exemple

```
set_opt_permissions("/opt/monapp") # /opt/monapp devient 755, et tout ce qu'il
contient (fichiers compris) aussi.
```

Erreurs à ne pas faire - Utiliser cette fonction sur un dossier contenant des fichiers sensibles avec des permissions volontairement restrictives (clés privées, fichiers de config avec des secrets en 600) : la fonction est **récursive sur tout, fichiers compris** (décision prise explicitement avec l'utilisateur, voir la conversation du 2026-07 sur `/opt`). Un fichier en 600 dans l'arborescence passerait en 755, donc lisible/exécutable par tout le monde. À réserver aux dossiers d'applications sans secret dedans. - Croire que 755 sur un fichier de données (pas un binaire) pose un souci de sécurité : ça rend le fichier "exécutable" au sens des bits Unix, ce qui est inoffensif pour un fichier texte (le bit x est juste ignoré s'il n'y a rien à exécuter), mais reste inhabituel et peut fausser des heuristiques d'outils tiers qui se fient au bit exécutable pour deviner le type de fichier.

Pourquoi ainsi `os.walk` plutôt que `pathlib.Path.rglob("*")` : les deux fonctionnent, mais `os.walk` donne directement `dossiers` et `fichiers` séparés à chaque niveau, ce qui évite d'avoir à tester `.is_dir()`/`.is_file()` sur chaque élément — plus direct pour ce cas d'usage précis (poser un chmod différent... ici identique, mais la structure reste plus lisible).

---

java_setup.py

`copy_java(java_src_dir, version_dirname, install_root="/opt/java")`

Syntaxe

```
copy_java(java_src_dir: str, version_dirname: str, install_root: str = "/opt/java") ->
str # -> chemin de destination, ex. "/opt/java/8.0.392-tem"
```

Utilité Copie récursivement `java_src_dir/version_dirname` (le JDK extrait) vers `install_root/version_dirname`. Renvoie le chemin final (= le futur `JAVA_HOME`).

Exemple

```
java_home = copy_java("/opt/install/java", "8.0.392-tem") # -> "/opt/java/8.0.392-tem"
```

Erreurs à ne pas faire - Construire ce dossier source sur un partage réseau qui ne supporte pas les liens symboliques (ex. un partage VMware `hgfs`) : un JDK contient des symlinks (`libjsig.so`, des pages de man localisées...), et `shutil.copytree` échouera à les recréer si le système de fichiers source ou destination ne les supporte pas. Voir l'incident documenté dans `INSTALL.md` — toujours construire/extraire ce genre d'arborescence sur un système de fichiers Linux natif. - Relancer `copy_java` en pensant qu'elle écrasera proprement une installation précédente corrompue : `dirs_exist_ok=True` **fusionne** avec ce qui existe déjà (les fichiers du même nom sont écrasés, mais les fichiers en trop qui ne sont plus dans la source ne

sont **pas** supprimés). Si tu changes de version de JDK sans changer `version_dirname`, il faut supprimer `install_root/version_dirname` toi-même avant de relancer.

Pourquoi ainsi `dirs_exist_ok=True` (plutôt que planter si le dossier existe déjà) permet de relancer `main.py` sur une machine déjà partiellement provisionnée sans avoir à nettoyer `/opt/java` à la main entre deux essais.

`configure_java_env(java_home, profile_path="/etc/profile.d/java.sh")`

Syntaxe

```
configure_java_env(java_home: str, profile_path: str = "/etc/profile.d/java.sh") ->
None
```

Utilité Écrit un script shell dans `/etc/profile.d/` qui exporte `JAVA_HOME` et ajoute `$JAVA_HOME/bin` en tête du `PATH`. Pose `chmod 644` dessus.

Exemple

```
configure_java_env("/opt/java/8.0.392-tem") # Écrit /etc/profile.d/java.sh : # export
JAVA_HOME=/opt/java/8.0.392-tem # export PATH=$JAVA_HOME/bin:$PATH
```

Erreurs à ne pas faire - S'attendre à ce que `java -version` fonctionne immédiatement dans le terminal **courant** juste après avoir lancé `main.py` : `/etc/profile.d/*.sh` n'est lu qu'au **login** d'une session shell (login shell). Un terminal déjà ouvert avant le run du script ne verra pas la variable tant qu'on n'a pas ouvert une nouvelle session (ou fait `source /etc/profile.d/java.sh` manuellement dans ce terminal précis). - Ajouter un `source /etc/profile.d/java.sh` dans le script Python en pensant que ça propagera la variable aux futures sessions des utilisateurs : un `subprocess.run(["source", ...])` (ou même un appel shell) ne changerait que l'environnement du sous-process lancé, jamais celui des sessions d'autres utilisateurs, présentes ou futures. C'est un piège classique en scripting shell/Python : chaque process a son propre environnement, hérité par ses enfants mais jamais partagé "vers le haut" ou "sur le côté".

Pourquoi ainsi `/etc/profile.d/` est l'emplacement standard pour ce genre de réglage global sous Linux (lu automatiquement par `bash`, `sh`, etc. au login), plutôt que d'éditer `/etc/environment` ou le `.bashrc` de chaque utilisateur — un seul fichier à gérer, appliqué à tous les comptes présents et futurs sans intervention supplémentaire.

`install_java(java_src_dir, version_dirname, install_root="/opt/java")`

Syntaxe

```
install_java(java_src_dir: str, version_dirname: str, install_root: str = "/opt/java")
-> None
```

Utilité Fonction "chef d'orchestre" : `copy_java` → `opt_permissions.set_opt_permissions` (chmod 755 récursif, réutilisé depuis le module 6) → `configure_java_env`. Seule fonction du module appelée depuis `main.py`.

Exemple

```
install_java("/opt/install/java", "8.0.392-tem")
```

Erreurs à ne pas faire - Appeler `configure_java_env` avant `set_opt_permissions` si jamais tu réorganises ce code : l'ordre actuel (copie → permissions → variables d'env) n'a pas de dépendance stricte entre permissions et variables d'env, mais il est cohérent avec la logique "d'abord les fichiers sont en place et lisibles, ensuite on les expose à l'environnement".

Pourquoi ainsi Réutilisation de `opt_permissions.set_opt_permissions` plutôt que de dupliquer un `chmod -R 755` ici : c'est exactement la même opération que pour les autres dossiers d'applis sous `/opt/`, pas de raison d'avoir deux implémentations du même comportement.

---

password_policy.py

`Constante` `REGLAGES`

Syntaxe

```
REGLAGES: dict[str, int] = { "minlen": 5, "difok": 0, "dcredit": 0, "ucredit": 0,
"lcredit": 0, "ocredit": 0, "minclass": 0, "maxrepeat": 0, "maxclassrepeat": 0,
"maxsequence": 0, "usercheck": 0, }
```

Utilité Valeurs par défaut appliquées à `/etc/security/pwquality.conf` : longueur minimale 5 caractères, toutes les autres contraintes de complexité désactivées (0 = pas de restriction), et `usercheck=0` pour autoriser le nom d'utilisateur comme mot de passe.

Erreurs à ne pas faire - Modifier ce dict en pensant que ça n'affecte qu'un run particulier : c'est une constante au niveau du module, partagée par tous les appels à `relax_password_policy()` qui ne précisent pas explicitement `reglages=...`. Pour un réglage ponctuel différent, passer un dict en paramètre plutôt que modifier `REGLAGES` en mémoire.

Pourquoi ainsi Toutes les valeurs à `0` plutôt que de simplement supprimer les lignes du fichier : `pwquality.conf` a ses propres valeurs par défaut internes si une clé est absente (souvent restrictives), donc "absent" ne veut pas dire "désactivé" — il faut explicitement écrire `0` pour être sûr du résultat.

`relax_password_policy(conf_path="/etc/security/pwquality.conf", reglages=None)`

Syntaxe

```
relax_password_policy(conf_path: str = "/etc/security/pwquality.conf", reglages: dict
| None = None) -> None
```

Utilité Modifie `conf_path` ligne par ligne : pour chaque clé de `reglages` (ou `REGLAGES` par défaut), remplace la ligne existante si elle est trouvée (commentée ou active), sinon l'ajoute à la fin du fichier. Ne touche à aucune autre ligne.

Exemple

```
relax_password_policy() # Avant : "# minlen = 8" # Après : "minlen = 5" # (les lignes
non concernées, ex. des commentaires explicatifs, restent inchangées)
```

Erreurs à ne pas faire - S'attendre à ce que cette fonction commente/décommente intelligemment : elle ne fait que repérer la clé via une regex (`^\s*#?\s*(\w+)\s*=`) et remplacer **toute la ligne** par `clé = valeur` sans le `#`. Si le fichier avait plusieurs lignes pour la même clé (une commentée, une active, ce qui est rare mais possible après des éditions manuelles successives), seule la **dernière rencontrée dans le fichier** aura le dernier mot (chaque itération de la boucle écrase l'entrée précédente à cette clé dans `lignes[i]`, mais toutes les lignes correspondantes sont réécrites - donc en pratique toutes finissent avec la bonne valeur, pas de bug, juste plusieurs lignes actives identiques dans le fichier final). - Lancer cette fonction sans avoir vérifié qu'elle tourne bien en root : elle écrit dans `/etc/security/`, un dossier protégé par défaut sur toute distribution Linux.

Pourquoi ainsi Édition ligne par ligne avec une regex plutôt qu'un écrasement complet du fichier : `pwquality.conf` contient souvent des commentaires explicatifs utiles (documentation inline fournie par le paquet `libpwquality`) qu'on veut préserver, seules les lignes de réglage nous intéressent.

---

deploy_bundle.py

Constante `DESTINATIONS`

Syntaxe

```
DESTINATIONS: dict[str, str] = { "libreoffice": "/opt/install/libreoffice", "rpms":
"/opt/install/rpms", "desktop": "/opt/install/desktop", "java": "/opt/install/java", }
```

Utilité Fait correspondre chaque sous-dossier attendu dans l'archive (`opt/install/<clé>/`) à son dossier de destination final sur la machine cible.

Pourquoi ainsi Un dict plutôt que trois variables séparées (`RPMS_DEST`, `DESKTOP_DEST`, `JAVA_DEST`) : `deploy_bundle` peut boucler dessus (`for nom_dossier, destination in DESTINATIONS.items()`) au lieu de répéter trois fois le même bloc de code pour chaque dossier.

`deploy_bundle(archive_path)`

Syntaxe

```
deploy_bundle(archive_path: str) -> None
```

Utilité Extrait l'archive tar (compressée ou non, peu importe l'extension) dans un dossier temporaire, puis pour chacun des trois sous-dossiers de `DESTINATIONS` : supprime la destination si elle existe déjà, puis copie fraîchement depuis l'extraction. Nettoie le dossier temporaire à la fin, même en cas d'erreur.

Exemple

```
deploy_bundle("/root/bundle_install.tar.gz") # opt/install/rpms/ -> /opt/install/rpms
# opt/install/desktop/ -> /opt/install/desktop # opt/install/java/ ->
/opt/install/java
```

Erreurs à ne pas faire - Interrompre le script (Ctrl+C) pendant l'extraction ou la copie : le `finally` garantit que le dossier temporaire est nettoyé, mais si l'interruption tombe pile pendant un `shutil.rmtree(destination)` (juste après avoir vidé une destination, avant d'avoir fini la recopie), tu peux

te retrouver avec un dossier de destination à moitié vide. Il vaut mieux relancer `deploy_bundle` en entier dans ce cas plutôt que de bricoler à la main. - Donner une archive dont la structure interne ne commence pas par `opt/install/` (ex. une archive créée avec `tar czf bundle.tar.gz rpms/ desktop/ java/` sans le préfixe `opt/install/`) : la fonction lèvera un `FileNotFoundError` explicite plutôt que d'échouer silencieusement, mais il faut reconstruire l'archive avec le bon préfixe (voir `INSTALL.md`, section construction du bundle).

Pourquoi ainsi Vider la destination avant de recopier (`shutil.rmtree` puis `shutil.copytree`) plutôt que de fusionner (`dirs_exist_ok=True` comme dans `java_setup.copy_java`) : ici, contrairement au JDK, on veut activement **empêcher** un mélange entre un ancien bundle RPM et un nouveau — c'est la leçon tirée de l'incident où un vieux lot de 518 RPM traînait à côté d'un nouveau lot propre et provoquait des conflits `dnf` à répétition.

---

install.py

`install(archive_path, phase="all")`

Syntaxe

```
install(archive_path: str, phase: str = "all") -> None # phase : "all", "provision" ou "accounts"
```

Utilité Point d'entrée unique du projet : déploie le bundle (`deploy_bundle(archive_path)`) puis lance `main(phase)` (importé depuis `main.py`). Évite d'avoir à lancer deux commandes séparées. Le paramètre `phase` choisit quelle partie de l'installation tourne (voir `main.py`).

Le déploiement du bundle n'a lieu que pour les phases `"all"` et `"provision"` : la phase `"accounts"` ne touche à aucun fichier du bundle (ni RPM, ni Java, ni raccourcis), donc il est inutile de tout vider et recopier si ça a déjà été fait pendant `"provision"`.

Exemple

```
install("/root/bundle_install.tar.gz") # tout, comme avant
install("/root/bundle_install.tar.gz", phase="provision") # déploiement + partie sans interaction install("/root/bundle_install.tar.gz", phase="accounts") # seulement les comptes
```

En ligne de commande :

```
sudo python3 install.py /root/bundle_install.tar.gz sudo python3 install.py /root/bundle_install.tar.gz --phase provision sudo python3 install.py /root/bundle_install.tar.gz --phase accounts
```

Le bloc `if __name__ == "__main__":` utilise `argparse` (module standard de lecture d'options en ligne de commande) : `bundle_path` est un argument obligatoire, `--phase` est optionnel avec `choices=[...]`, ce qui fait refuser automatiquement toute autre valeur avec un message clair, et `default="all"` garde l'ancien comportement quand on ne précise rien.

Erreurs à ne pas faire - Appeler `run_install(phase)` avec un `main()` qui n'accepte pas ce paramètre : `install.py` et `main.py` doivent évoluer ensemble. Une fusion de branches mal résolue a déjà produit exactement cette incohérence (`TypeError: main() takes 0 positional arguments but 1 was`

`given`). - Importer `main` avant que `deploy_bundle` ait tourné en pensant gagner du temps (ex. paralléliser les deux) : l'import Python (`from main import main as run_install`) n'exécute que les définitions du module, pas de souci à ce niveau — mais l'**appel** `run_install(phase)` doit impérativement se faire après `deploy_bundle(archive_path)`, sinon `main()` lira des fichiers RPM/desktop/java qui n'ont pas encore été placés par le déploiement. - Taper `--phase` avec une valeur inventée (`--phase tout`) : `argparse` s'arrête avec un message d'usage et un code d'erreur, rien n'est lancé.

Pourquoi ainsi Un alias `from main import main as run_install` plutôt qu'un simple `from main import main` : évite toute ambiguïté de lecture entre la fonction `main()` importée et la fonction `install()` définie dans ce même fichier — surtout utile si quelqu'un lit ce fichier sans avoir `main.py` ouvert à côté. `argparse` plutôt que lire `sys.argv` à la main : avec une option nommée et facultative (`--phase`), compter les arguments soi-même devient vite fragile, `argparse` gère l'ordre, les valeurs interdites et le message d'aide (`--help`).

---

main.py

`main.py` est découpé en trois fonctions : deux « moitiés » (`run_provisioning`, `run_accounts`) et un chef d'orchestre (`main`) qui choisit laquelle lancer.

`run_provisioning(cfg)`

Syntaxe

```
run_provisioning(cfg: dict) -> None
```

Utilité Toute la partie **sans interaction** : elle enchaîne, dans l'ordre, LibreOffice (lot RPM séparé, en premier, sa propre transaction `dnf`) → RPM habituels → Java → politique de mot de passe → raccourcis → extensions GNOME → permissions `/opt`. Elle lit les chemins dans `cfg` (le dict renvoyé par `load_config()`).

Cas particulier : le profil de durcissement Avant d'assouplir `pwquality.conf`, la fonction lit la variable d'environnement `OL96_HARDENING_PROFILE` :

```
hardening_profile = os.environ.get("OL96_HARDENING_PROFILE", "none") if
hardening_profile in ("", "none"): password_policy.relax_password_policy()
```

- Variable absente, vide ou égale à `none` : l'assouplissement a lieu, comme d'habitude.
- Variable définie avec un nom de profil (CIS, STIG...) : l'assouplissement est **sauté**, car OpenSCAP a déjà durci `pwquality.conf` et on ne veut pas écraser ce réglage. Un message l'indique à l'écran.

`os.environ.get("NOM", "valeur_par_défaut")` lit une variable d'environnement sans planter si elle n'existe pas : elle renvoie alors la valeur par défaut (ici `"none"`).

Erreurs à ne pas faire - Changer l'ordre des appels sans réfléchir aux dépendances entre étapes. LibreOffice passe en premier parce que ses RPM viennent de l'ISO 9.6 et collent à l'OS vierge : l'installer après les RPM habituels, qui peuvent mettre à jour certaines bibliothèques, ré-ouvrirait un risque de conflit. - Lancer sans `sudo` : chaque sous-fonction vérifie `os.geteuid() != 0` et lève une `PermissionError` dès la première étape concernée (l'échec est rapide et clair, mais autant lancer directement avec `sudo`).

Pourquoi ainsi Chaque étape est un appel direct à la fonction « chef d'orchestre » du module concerné (`install_libreoffice`, `install_rpms`, `install_java`, `enable_extensions`...), jamais aux fonctions

internes de bas niveau : `main.py` n'a pas besoin de connaître le détail de chaque module, seulement son point d'entrée public, ce qui permet de faire évoluer un module sans toucher à `main.py`.

`run_accounts()`

Syntaxe

```
run_accounts() -> None
```

Utilité La partie **interactive** : tant que l'opérateur répond `o` à « Créer un compte utilisateur ? », elle appelle `accounts.prompt_user_info()` (les questions) puis `accounts.create_user(**infos)` (la création).

Erreurs à ne pas faire - La lancer sans terminal interactif (script automatique, tâche planifiée) : `input()` attend une saisie qui ne viendra jamais, ou lève une `EOFError`. C'est précisément pour ça que cette partie est séparée de `run_provisioning`.

Pourquoi ainsi Les comptes ne passent pas par `config.json` : un mot de passe ne doit pas être écrit en clair dans un fichier. Les isoler dans leur propre fonction permet de les lancer à part (`--phase accounts`), par exemple à la console pendant que le reste tourne sans surveillance.

`main(phase="all")`

Syntaxe

```
main(phase: str = "all") -> None # phase : "all", "provision" ou "accounts"
```

Utilité Charge `config.json` puis lance la ou les parties demandées :

`phase`	`run_provisioning`	`run_accounts`
`"all"` (défaut)	oui	oui
`"provision"`	oui	non
`"accounts"`	non	oui

Exemple

```
# Lancé directement (phase "all") : # sudo python3 main.py # Ou importé et appelé
(comme le fait install.py) : from main import main main() # tout
main(phase="provision") # sans les comptes main(phase="accounts") # seulement les
comptes
```

Erreurs à ne pas faire - Oublier que `phase="all"` est une **valeur par défaut** : si on appelle `main()` sans argument, elle vaut `"all"`. C'est ce qui garde l'ancien comportement pour tout appel existant. - Faire accepter un paramètre `phase` à `install.py` sans le faire accepter à `main()` : `TypeError` immédiat.

Pourquoi ainsi Le `if phase in ("all", "provision")` plutôt que deux fonctions `main` séparées : un seul point d'entrée à maintenir, et la valeur par défaut `"all"` garantit que les scripts écrits avant l'arrivée des phases continuent de fonctionner sans modification.